



Lab Assignment # 02

Title:

Solving the Maze Using BFS Center

Submitted to :

- **Instructor** : Ma'am Khaula Qadeer
- **Course** : Artificial Intelligence(CSC-262)

Submitted by:

- **Name** : Sannan Ahmad
- **Reg #** (SP25-BSE-118)
- **Section** A

Date of Submission:

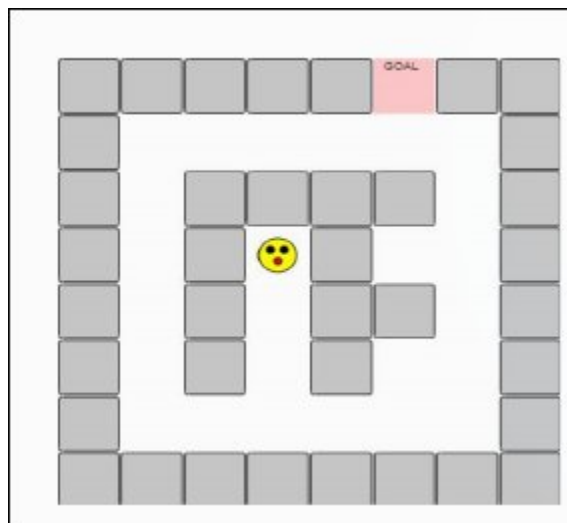
30 September, 2026

TABLE OF CONTENTS

Problem Statement :	2
Source Code :	2
Output :	6

Problem Statement :

Consider a maze as shown below. Each empty tile represents a separate node in the graph. There are a maximum of four possible actions i.e., to move up, down, left or right on any given tile/node. Using BFS, find out how to get out of the maze if you're in the start position depicted below.



Source Code :

```
#AI LAB ASSIGNMENT # 2
```

```
#SUBMITTED TO : MA'AM KHAULA QADEER
```

```
#NAME : SANNAN
```

```
#REG # SP25-BSE-118
```

```
### 1-> Closed Path,0->Open path,2->Start,3->Goal
```

```
maze = [
```

```
    [1, 1, 1, 1, 1, 3, 1, 1],
```

```
    [1, 0, 0, 0, 0, 0, 0, 1],
```

```
    [1, 0, 1, 1, 1, 1, 0, 1],
```

```
    [1, 0, 1, 2, 1, 0, 0, 1],
```

```
    [1, 0, 1, 0, 1, 1, 0, 1],
```

```
    [1, 0, 1, 0, 1, 0, 0, 1],
```

```
    [1, 0, 0, 0, 0, 0, 0, 1],
```

```
    [1, 1, 1, 1, 1, 1, 1, 1],
```

```
]
```

```
state_space = {}
```

```
start_state = None
```

```
goal_state = None
```

```
for r in range(len(maze)):
```

```
    for c in range(len(maze[0])):
```

```
        if maze[r][c] == 1:
```

```
            continue
```

```

if maze[r][c] == 2:

    start_state = (r, c)

if maze[r][c] == 3:

    goal_state = (r, c)


state_space[(r, c)] = []

for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:

    nr, nc = r + dr, c + dc

    if 0 <= nr < len(maze) and 0 <= nc < len(maze[0]) and maze[nr][nc] != 1:

        state_space[(r, c)].append((nr, nc))


def bfs(graph, start_node, end_node):

    frontier = []

    visited = []

    parent = {}


    frontier.append(start_node)

    visited.append(start_node)

    parent[start_node] = None


    while frontier:

        selected_node = frontier.pop(0)    # dequeue (FIFO)

```

```
if selected_node == end_node:
```

```
    path = []
```

```
    while selected_node is not None:
```

```
        path.append(selected_node)
```

```
        selected_node = parent[selected_node]
```

```
    path.reverse()
```

```
    return path
```

```
for neighbour in graph[selected_node]:
```

```
    if neighbour not in visited:
```

```
        frontier.append(neighbour)
```

```
        visited.append(neighbour)
```

```
        parent[neighbour] = selected_node
```

```
return None
```

```
path = bfs(state_space, start_state, goal_state)
```

```
if path:
```

```
    print("Path:", path)
```

```
    print("Total moves:", len(path) - 1)
```

```
else:
```

```
    print("No path found")
```

Output :

```
• [sannan@Ghost Lab Assignment 2]$ python Maze.py
Path: [(3, 3), (4, 3), (5, 3), (6, 3), (6, 4), (6, 5), (5, 5), (5, 6), (4, 6), (3, 6), (2, 6), (1, 6), (1, 5), (0, 5)]
Total moves: 13
❖ [sannan@Ghost Lab Assignment 2]$ |
```

FIN !